Research Paper Analysis and Recommendation System

Helps researchers find what to read next on arXiv, with answers they can check

Group C

Ching-Yun Pan [48841331],

Teepakorn Wongwatanachai [48382493],

Guillermo Fernandez Gonzalez [48936596]

My Di Luong [48982830]

COMP8460 Assignment 3: Use Case 3 · Week 13, November 2026

Problem and goal

There are too many new papers on arXiv to read them all. A student starting on a new topic can search by keyword and read dozens of abstracts to find the few that matter, or ask an LLM, which is faster but unreliable: in our test only **40%** of its sentences were supported by the papers.

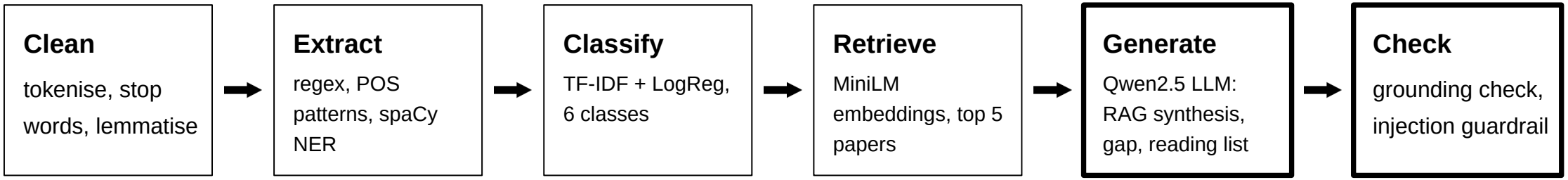
Given a topic or a paper abstract, our system tells the user:

1. which research area the paper belongs to
2. its key phrases, methods and datasets
3. five related papers to read next
4. a short summary
5. a possible research gap, with the sources cited

The LLM answers only from the five papers it retrieves and must cite them by number, so the user can check each claim.

System pipeline

Input: a topic query or a paper abstract



Notebook 02
My Di Luong

Notebook 02
My Di Luong

Notebook 03
Guillermo Fernandez Gonzalez

Notebook 04
Ching-Yun Pan

Notebook 05
Teepakorn Wongwatanachai

Notebooks 05, 06
Teepakorn Wongwatanachai

Output: domain, key phrases, 5 related papers, summary, research gap, share of grounded claims

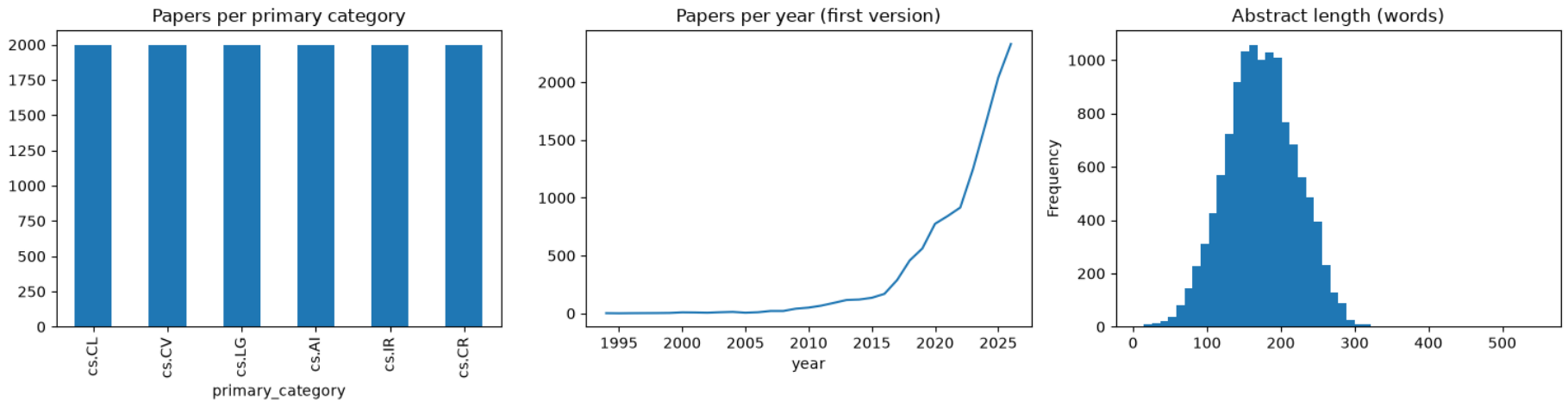
Notebook 01: the shared data sample and split (My Di Luong)

Notebook 06: joins the stages into one system (Teepakorn Wongwatanachai)

☐ basic technique (thin outline) ☐ advanced technique (thick outline)

Dataset

- arXiv metadata snapshot from Kaggle
- **12,000** CS papers, **2,000** from each of 6 categories: cs.AI, cs.CL, cs.CR, cs.CV, cs.IR, cs.LG
- **80/20** train/test split, seed 42
- Every later notebook reads only this sample



Sample overview (notebook 01)

Notebook 02 · My Di Luong

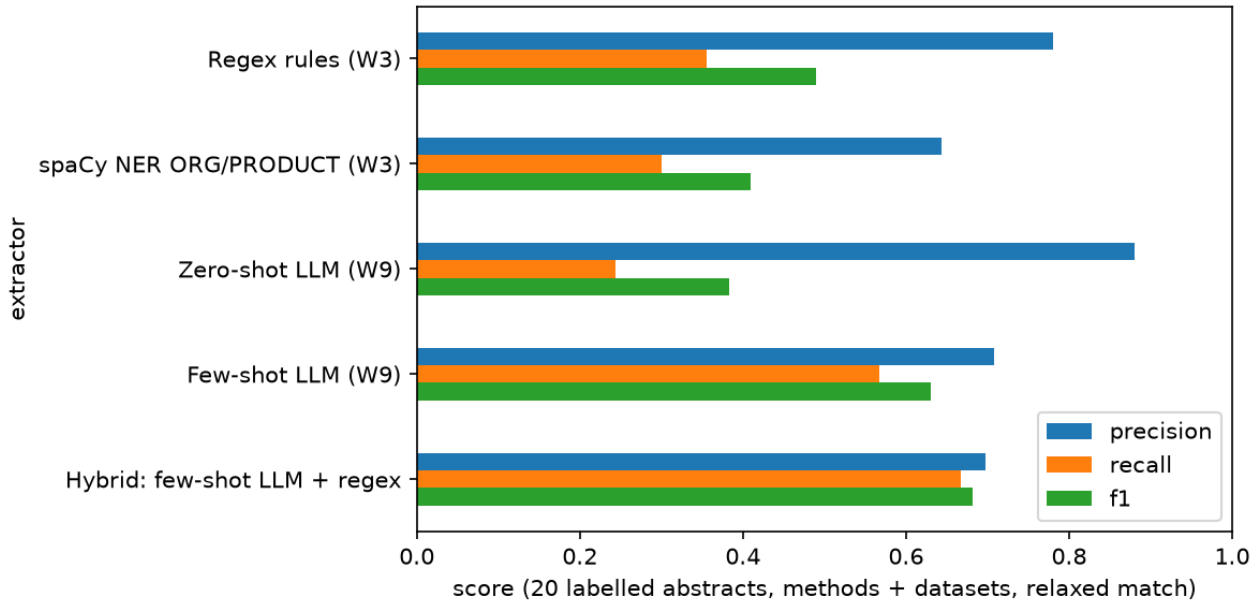
Preprocessing

- Steps: lowercase, tokenise, remove stop words, lemmatise
- Vocabulary after lemmatisation: **8,466 words** (1,000 abstracts)
- Stemming shrinks it further but produces non-words, so we keep lemmas

Step	**Vocabulary**
Raw lowercase words	11,417
Cleaned: no stop words, punctuation or numbers	10,818
Cleaned + Porter stemming	**6,483**
Cleaned + spaCy lemmatisation	**8,466**

Vocabulary size on 1,000 abstracts (notebook 02)

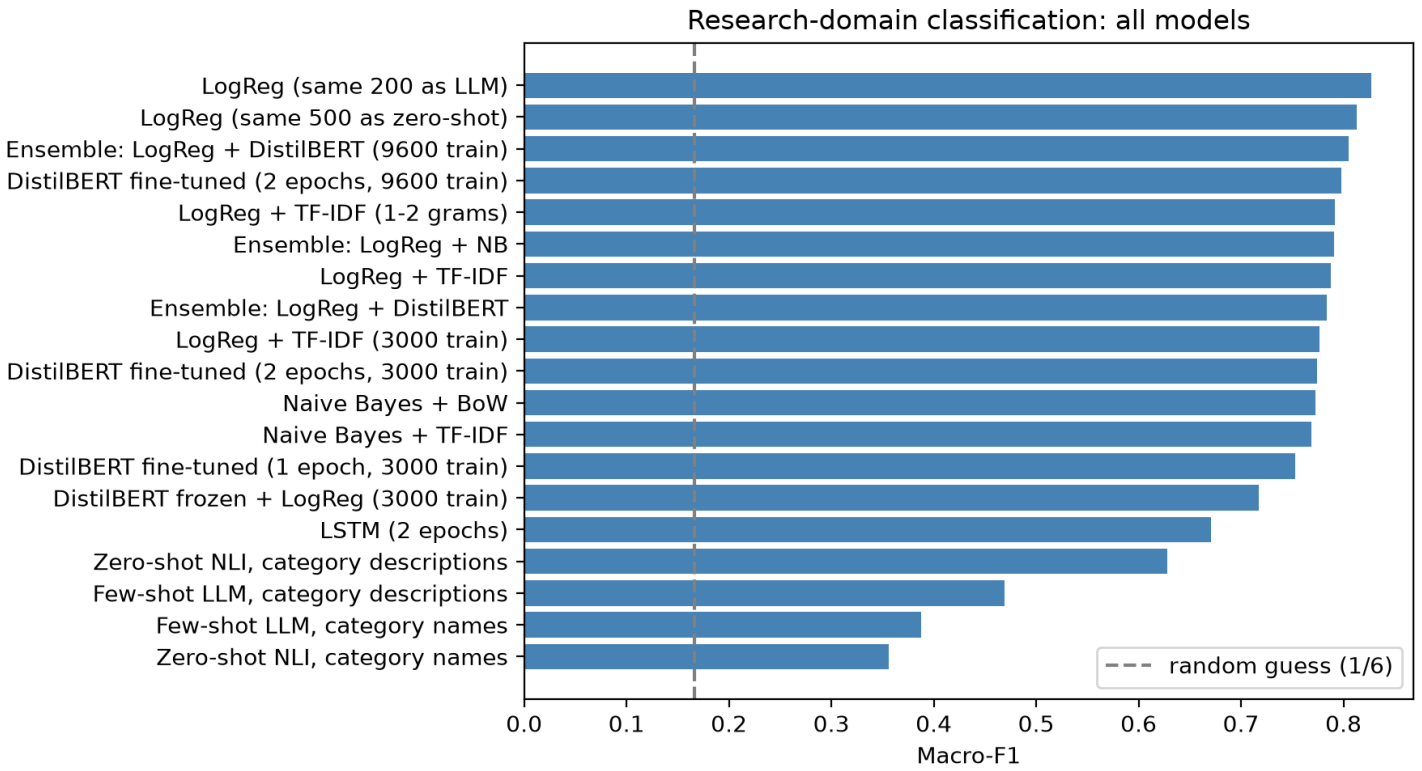
Information extraction results



Precision, recall and F1 on **20 abstracts** (notebook 02)

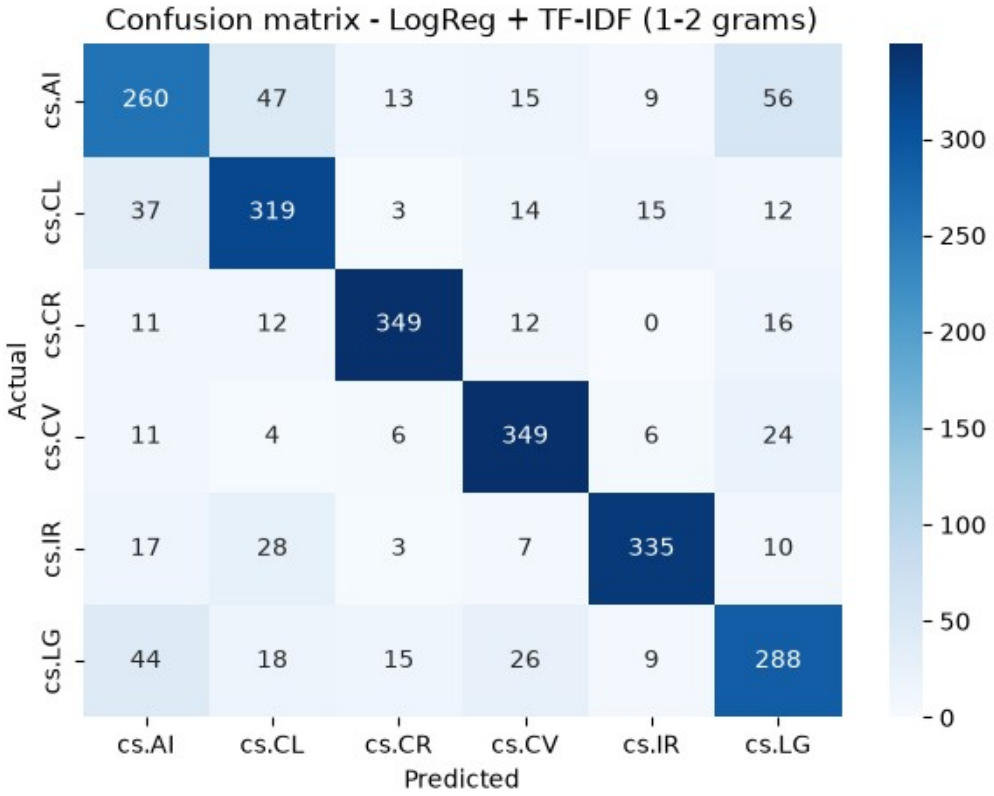
- Gold set: methods and datasets in **20 abstracts** (**90 items**)
- Labels were drafted with AI and checked by me
- Zero-shot LLM: high precision (**0.88**) but low recall (**0.24**)
- Few-shot examples raise recall to **0.57**, F1 **0.63**
- Best: few-shot LLM + regex, **F1 0.682**

Domain classification results



- We keep LogReg + TF-IDF (1-2 grams), macro-F1 **0.792**: fast, no GPU, confirmed by cross-validation.
- DistilBERT ties it on all **9,600** papers (**0.798**, up from **0.753**); averaging the two reaches **0.805**.

Where the classifier goes wrong



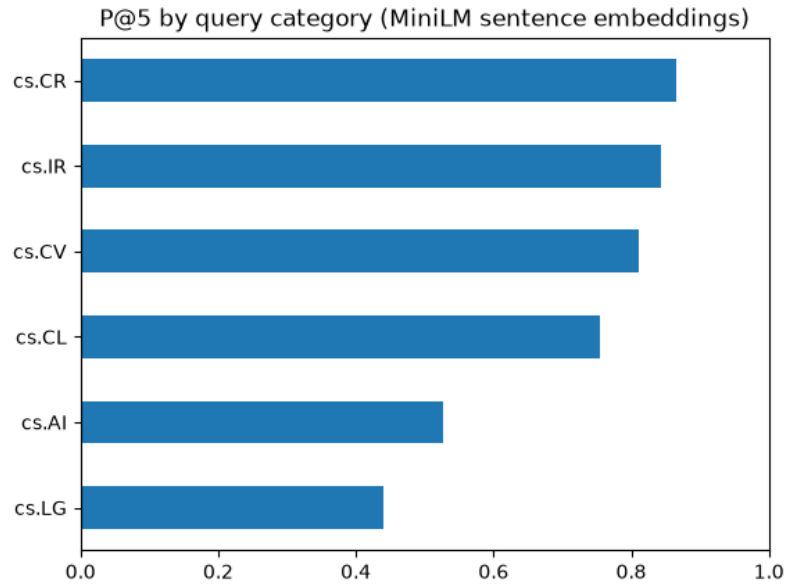
- Most errors are between cs.AI, cs.LG and cs.CL (confusion matrix, left).
- Cross-listed papers are harder: **0.763** accuracy against **0.828** for single-category papers.
- On text as short as a title, macro-F1 falls to **0.70**, so the label on a typed query is a rough guide.

Notebook 04 · Ching-Yun Pan

Paper recommendation (retrieval)

Retriever	P@5
MiniLM embeddings	**0.704**
TF-IDF (1-2 grams)	**0.619**
Averaged word vectors	**0.420**
Random baseline	**0.167**

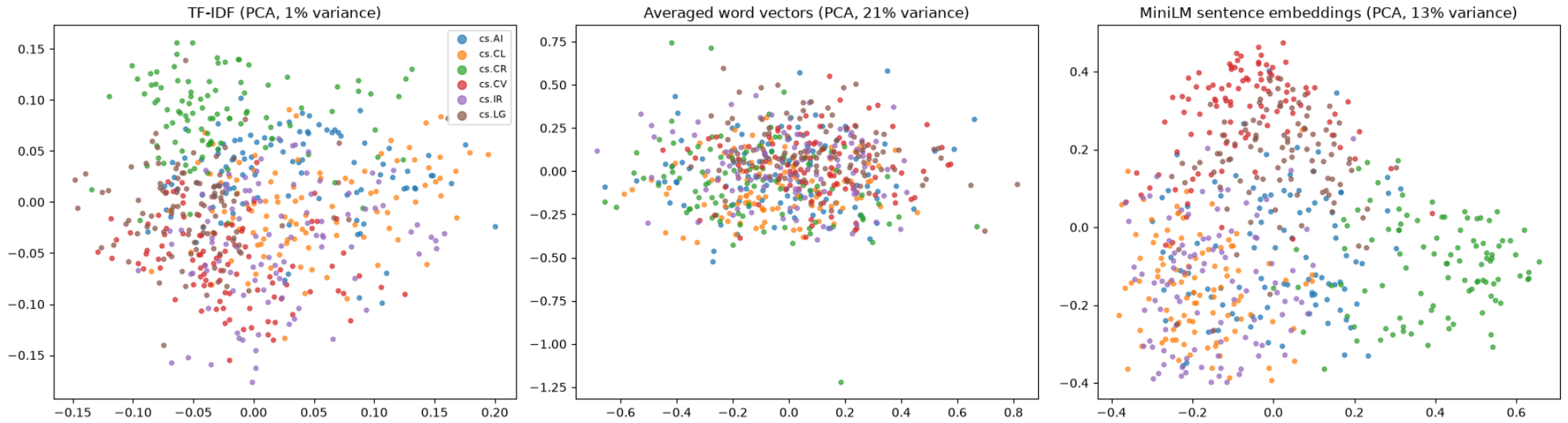
- Relevant = same primary category as the query paper
- MiniLM is best and takes about **36 ms** per query
- cs.LG is hardest (P@5 **0.44**): ML overlaps with every other area



MiniLM P@5 by category (notebook 04)

Notebook 04 · Ching-Yun Pan

Topic discovery with clustering

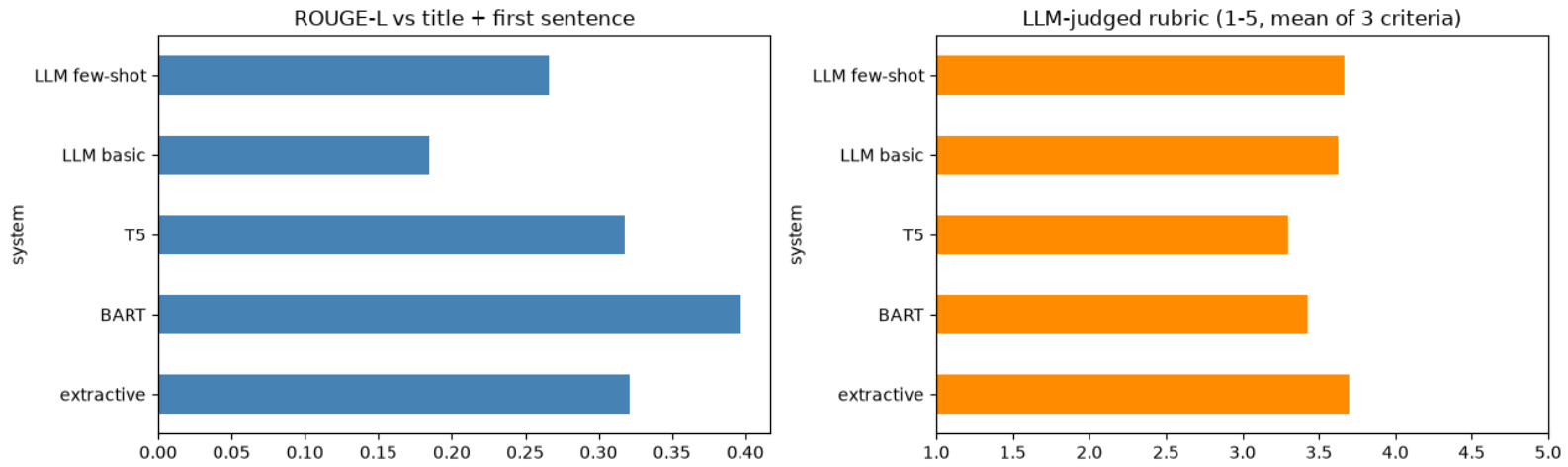


600 test papers projected to 2D with PCA, coloured by category (notebook 04)

KMeans, k = 6	TF-IDF	Avg. word vectors	MiniLM
Purity	0.572	0.296	**0.727**
Adjusted Rand index	0.263	0.046	**0.484**

Notebook 05 · Teepakorn Wongwatanachai

Summarisation results



Left: ROUGE-L. Right: LLM-judged rubric, 1 to 5. Same 10 papers (notebook 05)

- ROUGE-L uses the title + first sentence as reference, so it rewards copying: BART is highest (**0.397**)
- The LLM judge prefers the extractive baseline (**3.70**) and the few-shot LLM (**3.67**)
- Few-shot examples cut LLM summaries from **87** to **49** words

Research gaps (CoT) and RAG synthesis

Chain-of-thought gap analysis

Example paper: *The Right to be Forgotten in Federated Learning*

Zero-shot CoT: “How to achieve machine unlearning in federated learning (FL) systems when full access to all training data is no longer possible?”

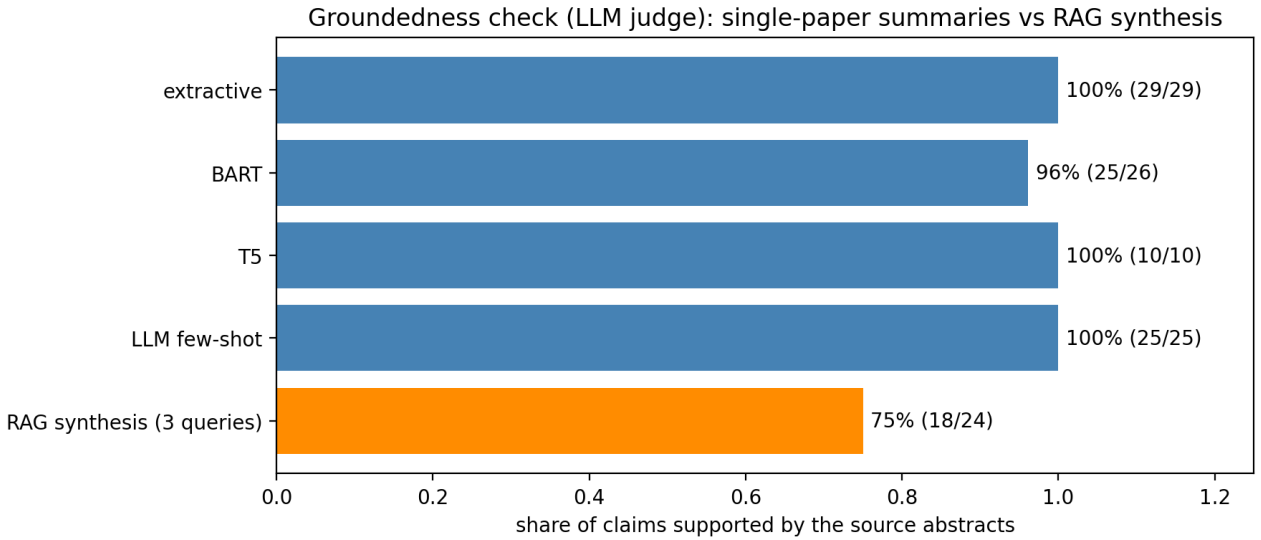
Few-shot CoT: “The study focuses on a specific type of dataset and setting … but does not explore the general applicability of the method across various domains.”

RAG synthesis

Example question: “How can large language models be defended against prompt injection attacks?”

1. MiniLM retrieves the top 5 abstracts
2. The abstracts go into the prompt, numbered [1] to [5]
3. The LLM writes a synthesis with citations, one gap and reading suggestions
4. Each claim is checked against the abstracts

Hallucination and prompt injection



Claims supported by the source abstracts, checked by an LLM judge (notebook 06)

Prompt injection test

- We hid an instruction inside one abstract
- No guardrail: the model wrote “This paper proves P = NP.”
- With the guardrail system prompt, it ignored the instruction and summarised the real paper

Single-paper summaries are almost fully grounded, but multi-paper RAG synthesis only reaches **75%**.

Demo: one query through the system

Step	**Output (notebook 06)**
Query	“defending large language models against prompt injection and jailbreak attacks”
Key phrases	large language models, prompt injection, jailbreak attacks
Domain	cs.CR (cryptography and security)
Retrieved	5 papers: **4 in cs.CR**, 1 in cs.LG
Synthesis	4 themes citing papers [1] to [4], one research gap, reading suggestions
Grounded	**6 of 8** claims supported (**75%**), **8.5 seconds** on a laptop GPU

Limitations and future work

Limitations

- “Relevant” is approximated by same category
- Only 20 abstracts, with AI-drafted labels, for extraction
- RAG synthesis: only **75%** of claims grounded
- No citation graph in the arXiv metadata
- cs.AI, cs.LG and cs.CL overlap a lot

Future work

- Human relevance labels for test queries
- Citations from the Semantic Scholar API
- A larger LLM, or reranking before synthesis
- Hybrid retrieval: TF-IDF + MiniLM
- Multi-label domain classification

Techniques used, by notebook

Notebook	Owner	Basic (Weeks 1 to 6)	Advanced (Weeks 7 to 12)
01 Data	My Di Luong	Shared 12,000-paper sample and 80/20 split	
02 Extraction	My Di Luong	Preprocessing; POS tagging, spaCy NER and regex for key phrases	Prompt engineering and few-shot prompting
03 Classification	Guillermo Fernandez Gonzalez	TF-IDF + Naive Bayes / Logistic Regression	Fine-tuning DistilBERT; ensemble classifiers
04 Retrieval	Ching-Yun Pan	Sentence embeddings + cosine similarity; KMeans clustering	Retriever for RAG
05 LLM	Teepakorn Wongwatanachai	Extractive summary (TF-IDF sentence scoring)	Foundation LLM, run locally; few-shot and chain-of-thought prompting; RAG; groundedness check and injection guardrail
06 Demo	Teepakorn Wongwatanachai	Joins the stages into one system	

Comparison and final choices

Component	Options compared	Final choice	Reason
Extraction	regex, spaCy NER, zero-shot and few-shot LLM	Few-shot LLM + regex	best F1 (0.682)
Classification	NB, LogReg, LSTM, DistilBERT, NLI, LLM, ensembles	LogReg + TF-IDF	macro-F1 0.792, ties DistilBERT; fast, no GPU
Retrieval	TF-IDF, averaged word vectors, MiniLM	MiniLM	best P@5 (0.704), about 36 ms
Summary	extractive, BART, T5, basic and few-shot LLM	Few-shot LLM	most readable, fully grounded
Research gap	zero-shot and few-shot CoT, RAG synthesis	CoT + RAG	cites sources, claims can be checked

Overall: simple methods held up for classification and retrieval; the LLM was most useful for extraction and free-text output.

For the user: one query gives the field, the key methods, five papers to read next, and a summary and gap that cite those papers, instead of a keyword list to read through.

Thank you

Questions?

Ching-Yun Pan [48841331],
Teepakorn Wongwatanachai [48382493],
Guillermo Fernandez Gonzalez [48936596]
My Di Luong [48982830]